Submission Worksheet

Submission Data

Course: IT202-010-S2026
Assignment: IT202 Milestone 1 - 2026
Student: Ryan C. (rc728)
Status: Graded | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 10.00/10.00 (100.00%)
Started: 4/12/2026 5:14:36 PM
Updated: 4/13/2026 11:43:31 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-1-2026/grading/rc728
View Link: https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-1-2026/view/rc728

Instructions

- Overview Link: https://youtu.be/IDtqdaLwcVg

1. Refer to Milestone1 of this doc: https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
    1. `git checkout Milestone1` (ensure proper starting branch)
    2. `git pull origin Milestone1` (ensure history is up to date)
5. Fill out the below worksheet
    - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    - Ensure proper styling is applied to each page
    - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone1`
    4. On Github merge the pull request from `Milestone1` to `qa`
    5. On Github create a pull request from `qa` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout qa`
11. `git pull origin qa`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

To Register A New Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

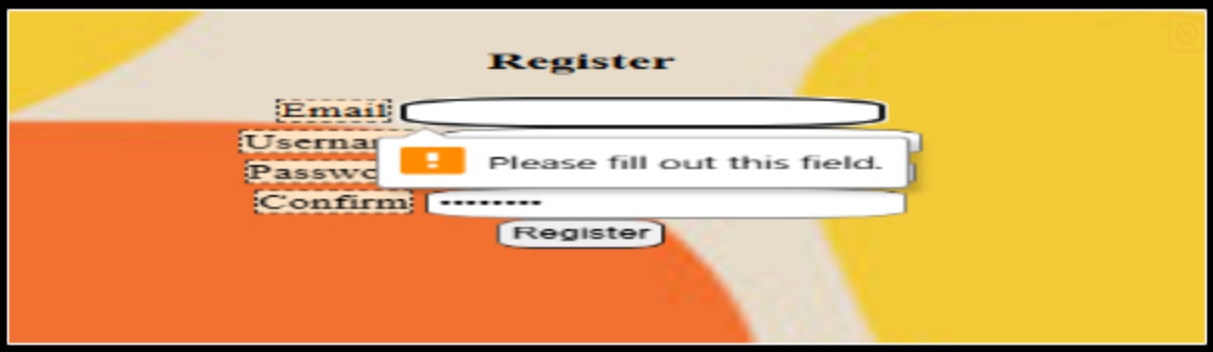
- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

this shows an email error

```
<h1>Register</h1>
<form onsubmit="return validate(this)" method="POST">
    <div>
        <label for="email">Email</label>
        <input id "email" type "email" name "email" required value "<?php echo htmlspecialchars($email ?? ''); ?>" />
    </div>
    <div>
        <label for="username">Username</label>
        <input type="text" name="username" required maxlength="30" value="<?php echo htmlspecialchars($username ?? '') ?>" />
    </div>
    <div>
        <label for="pw">Password</label>
        <input type="password" id="pw" name="password" required minlength="8" />
    </div>
    <div>
        <label for="confirm">Confirm</label>
        <input type="password" name="confirm" required minlength="8" />
```

this shows the code which allows for the username and email to stay after an error

invalid password error



empty field error

Saved: 4/13/2026 11:20:25 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The register form uses both HTML and PHP validation to be able to prevent errors. The HTML validation to be able to check the input before it is submitted and give feedback. The php validation is used on the server side to be able to check for email format, username rules, password length, matching passwords, or duplicate accounts. To keep be able to keep the form information still inside without wiping it after an error uses sticky form logic which was implemented into the email and the username field. It uses value="" /> for the email and value="" /> for the username. This allows for the user's input to stay in the form after the error allowing the user to correct mistakes without needing to type back all the information.

Saved: 4/13/2026 11:20:25 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

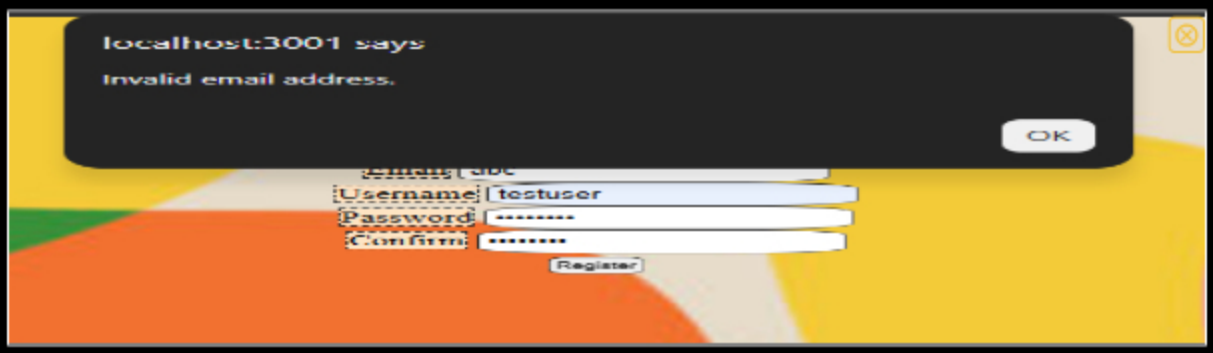
Progress: 100%

Part 1:

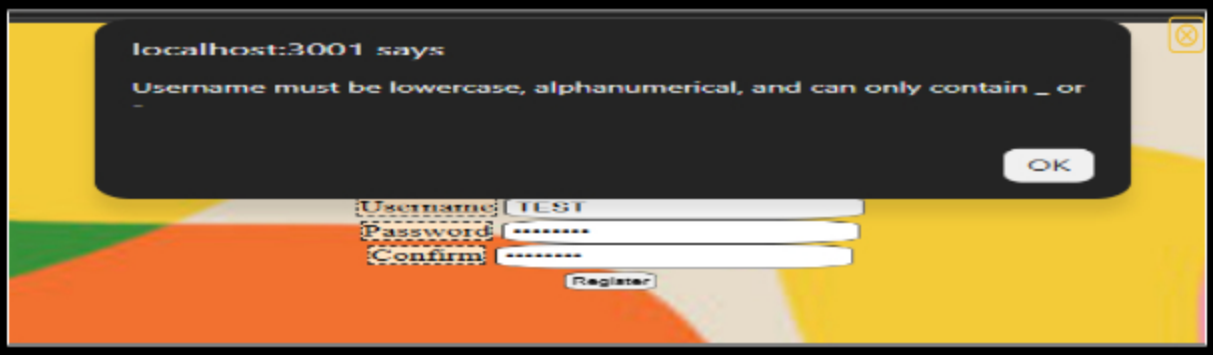
Progress: 100%

Details:

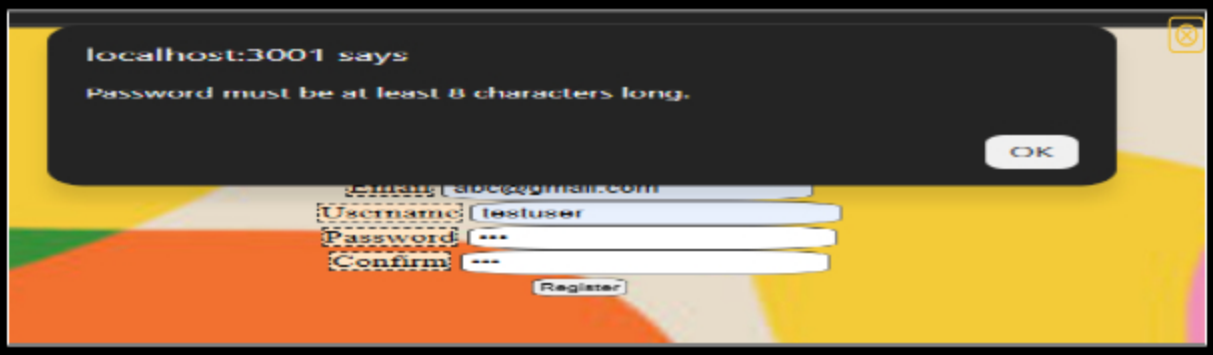
- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



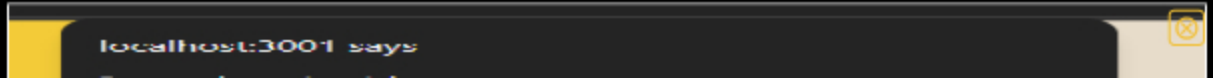
invalid email error

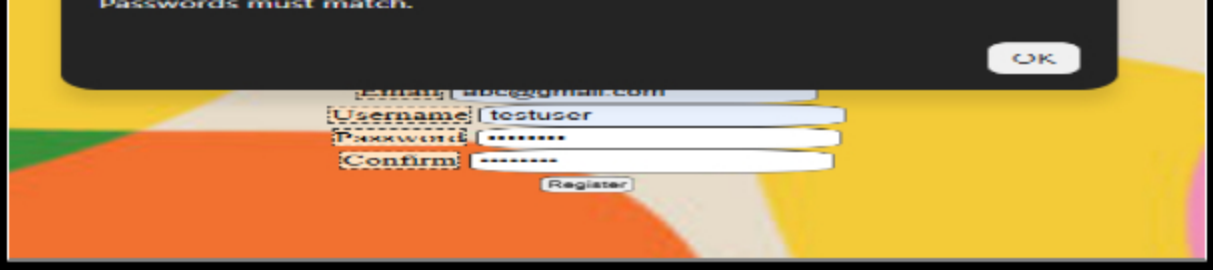


user must be lowercase error

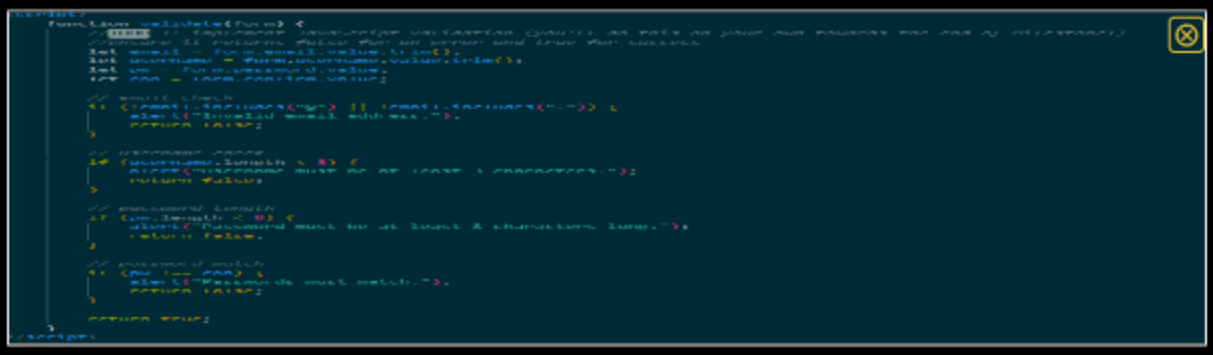


password length error





password must match error



this shows the js validation code

Saved: 4/13/2026 11:11:17 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validate() function uses JS validation before the form is submitted where it checks for valid email format, username format, password length, and if the passwords are matching. If any of these checks fails, there will be an alert pop up shown and the form submission is stopped. the allows for errors to be caught before data is sent to the server.

Saved: 4/13/2026 11:11:17 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

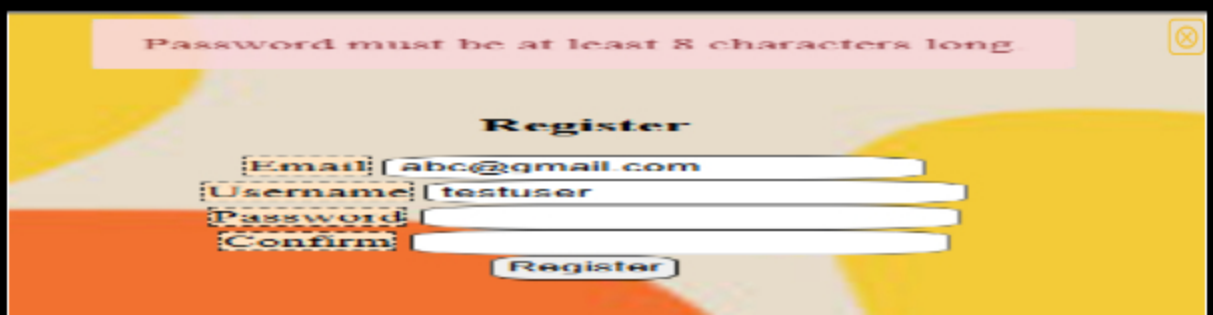
Details:

- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use

this shows the code for the php validation

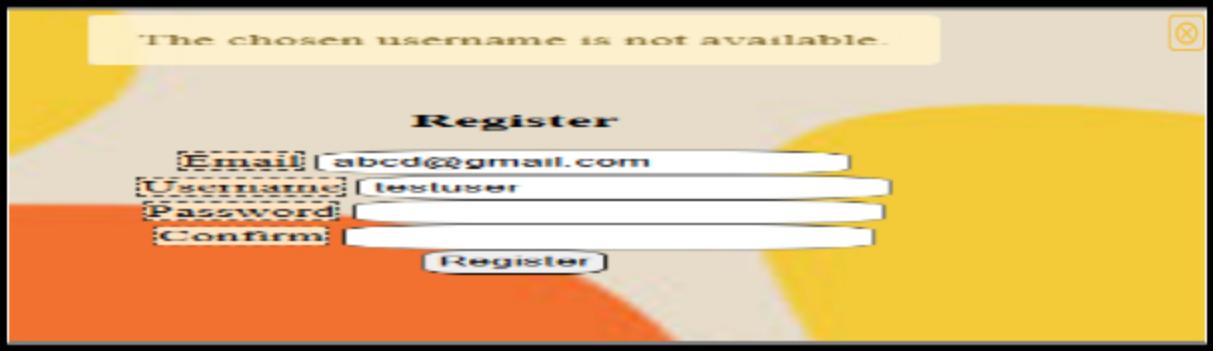
invalid email address error

username must be lowercase error

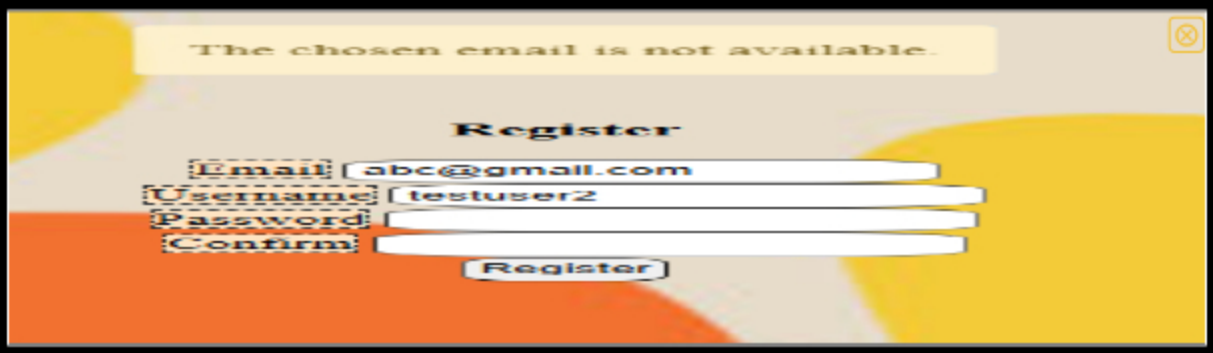


password must be at least 8 characters error

password must match error



username not available error



email not available error

Saved: 4/13/2026 11:11:14 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The register page uses PHP validation before sumbittion to check for valid email format, username format, password length, empty fields, and matching passwords.t also checks for

duplicate usernames and emails using database constraints. If any validation fails, a flash message is displayed and the user is not inserted into the database. This allows for invalid or duplicate data to be caught on the server before being stored.

Saved: 4/13/2026 11:11:14 PM

Task #2 (0 / 0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php`, , zero credit if it does not)
- Include the render.com prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/524RC/RC728-IT202-010/blob/prod/public_html/project/register.php
URL
https://github.com/524RC/R(

URL #2
https://rc728-it202-010-prod.onrender.com/project/register.php
URL
https://rc728-it202-010-prod.

URL #3
https://github.com/524RC/RC728-IT202-010/pull/23
URL
https://github.com/524RC/R(

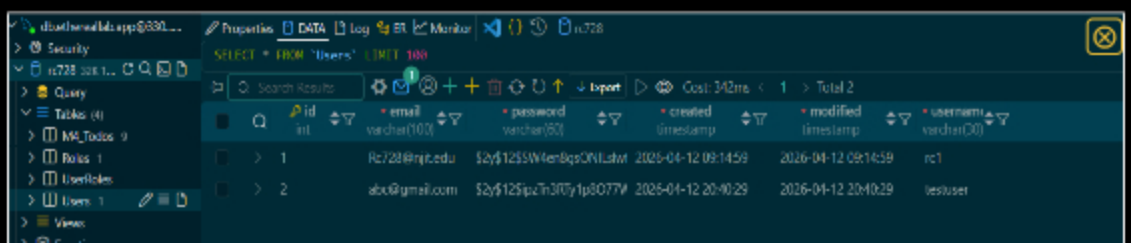
Saved: 4/13/2026 11:41:53 PM

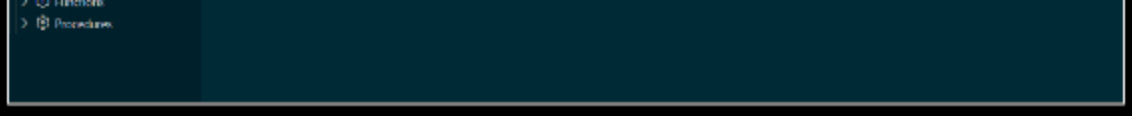
Task #3 (0 / 0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)





this shows the database

Saved: 4/13/2026 11:43:31 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 100%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

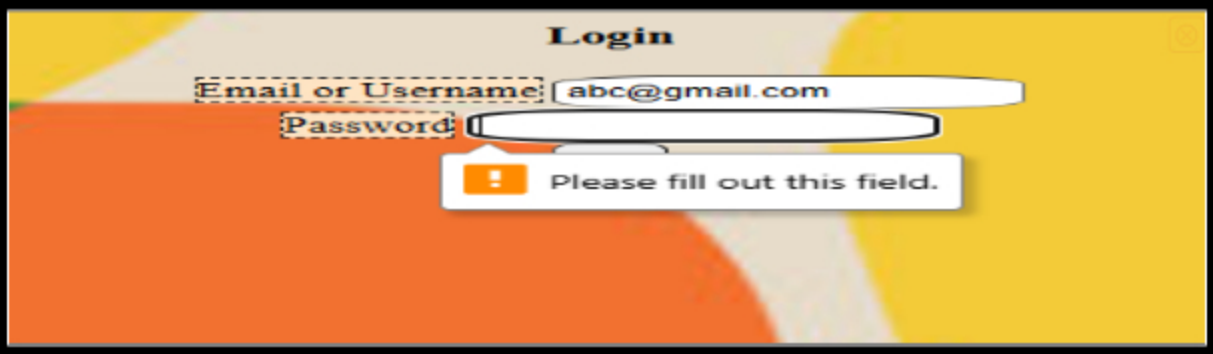
- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```
<h3>Login</h3>
<form onsubmit="return validate(this)" method="POST" >
    <div>
        <label for="email">Email or Username</label>
        <input id="email" type="text" name="email" required value="<?php echo htmlspecialchars($_POST['email'] ?? ''); ?>" />
    </div>
    <div>
        <label for="pw">Password</label>
```

```
    </div>
    <input type="submit" value="Login" />
</form>
```

this shows the html validation code

This shows an missing username or email error



this shows a missing password error

this shows a too short password error

Saved: 4/13/2026 3:13:17 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The HTML validation works by using the required attribute on both the username/email and password field to make it so the user cannot submit the form if either field is empty. The password field also includes that minlength="8" to make it so the minimum password length is 8. These validations are handled by the browser and display an error message. This allows the user to fix mistakes before submission.

Saved: 4/13/2026 3:13:17 AM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

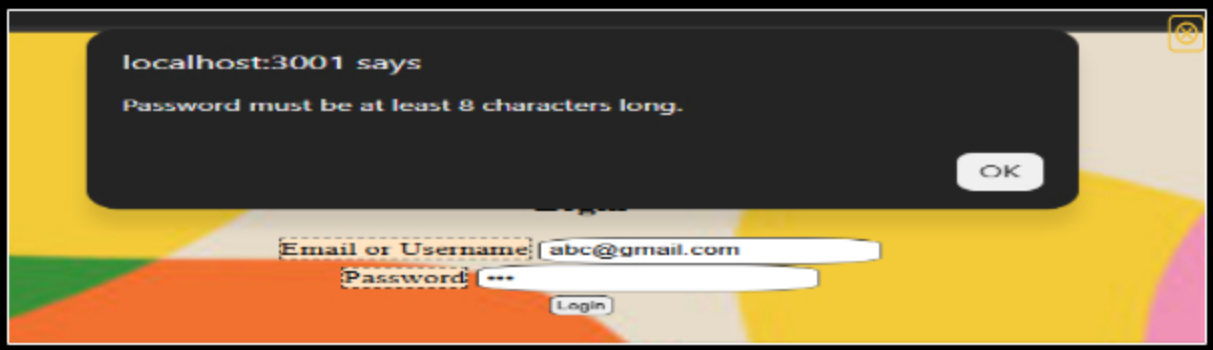
Part 1:

Progress: 100%

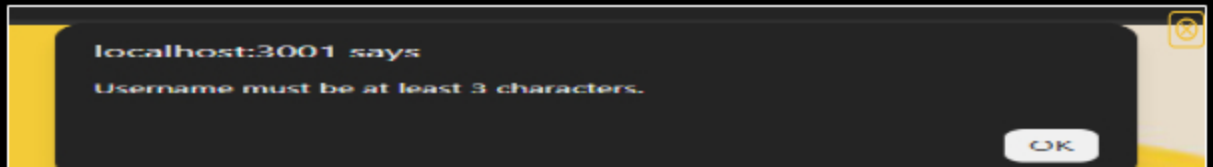
Details:

- Show the code related to the login page's validate() function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

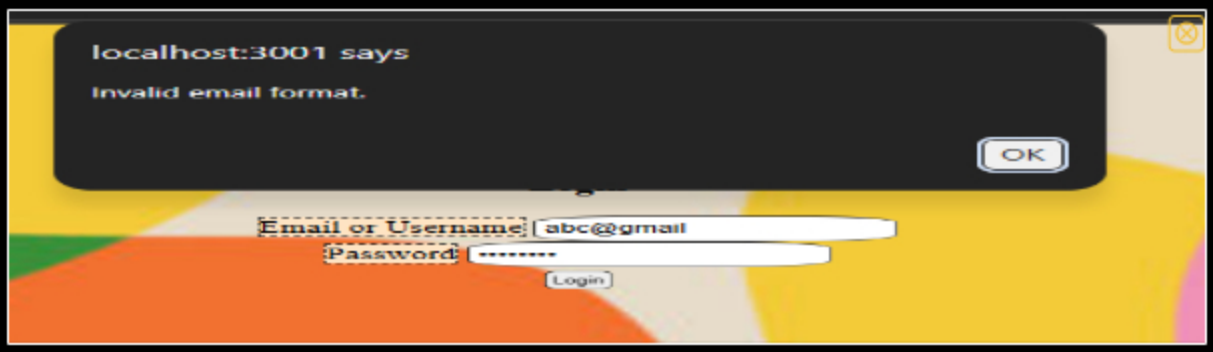
this shows the js validation



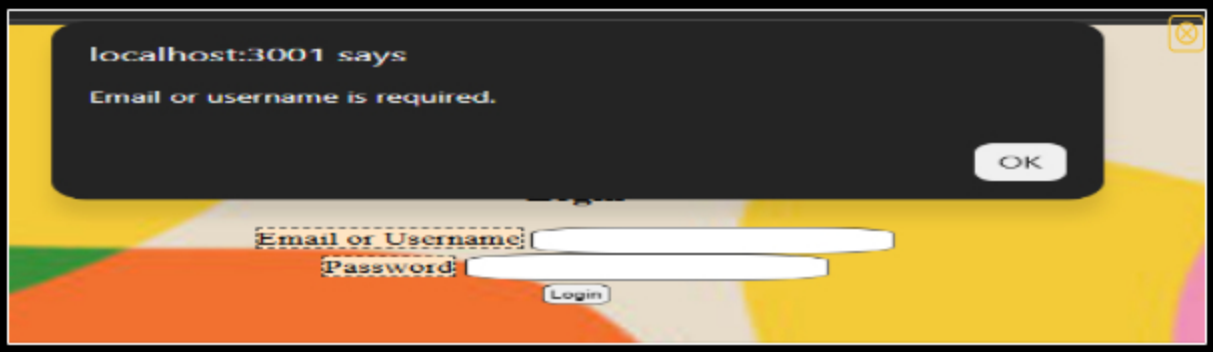
This shows an invalid password error



this shows an Invalid username error



this shows an Invalid email error



this shows an empty password and username/email error

Saved: 4/13/2026 8:53:44 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The JS validation is handled by the validate() function which would run when a form is submitted. It checks if the email/username field is empty, if the input is a valid email format if it has a @ and a valid username length, and if the password is atleast 8 characters long. if any of the checks fail, then an alert message is shown and the form submission is cancelled. This solves the requirement by validating user input on the client side and allowing users to fix errors before the form is submitted.

Saved: 4/13/2026 8:53:44 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

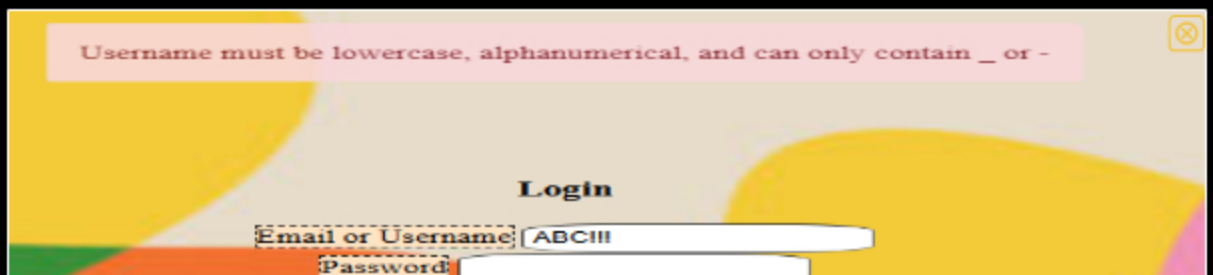
Progress: 100%

Details:

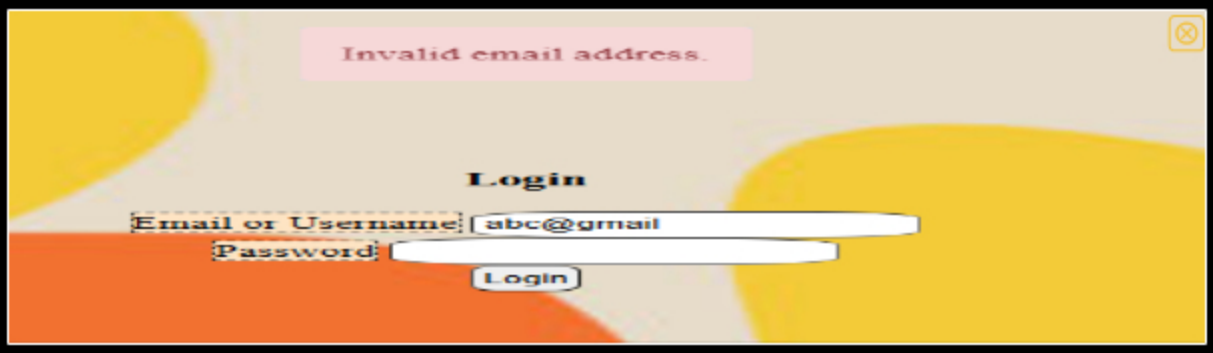
- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch (the one that checks against the DB hash)

this shows the code for the php validation

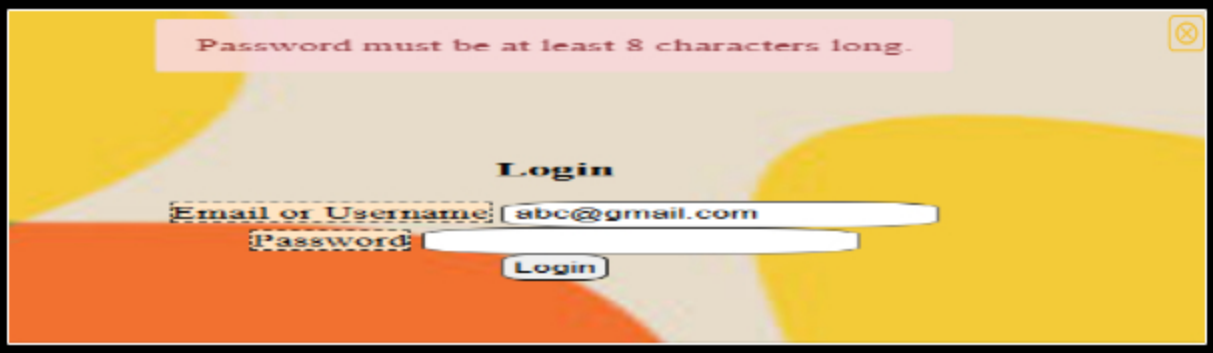
this shows the code for the php validation after the DB call for incorrect passwords and non existent users



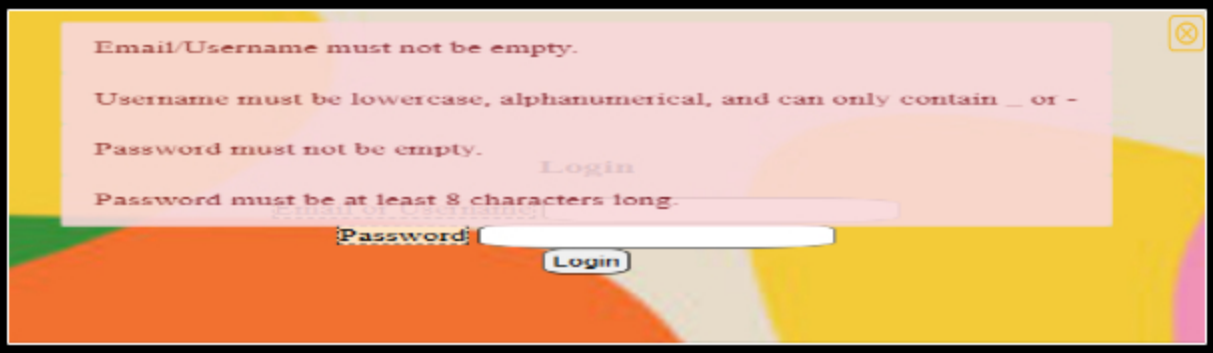
invalid username format error



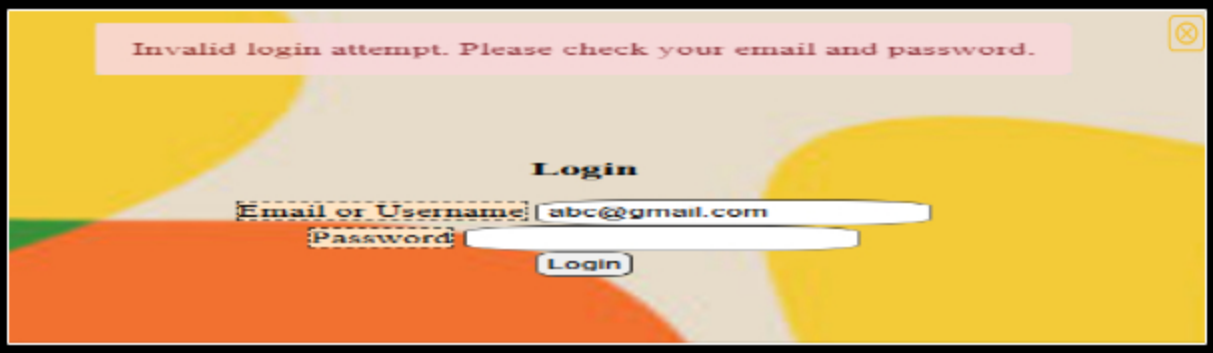
invalid email format error



invalid password error



empty fields error



invalid login attempt error

Saved: 4/13/2026 3:57:15 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

the PHP validations runs after the form is submtited and before the login is completed. it checks whether the email/username field is empty, if the input should be treated as an email or a username, and if it is following the correct format. The password also validates if the input is atleast 8 characters long and if any of these validations fail, a flash message will be displayed and the login process is stopped. After validation, the system will check the database to see if the user exists and if the password matches using the function password_verify(). If either the user is non-existent or the password was incorrect, the both will return the same error message for security.

Saved: 4/13/2026 3:57:15 AM

Task #2 (0 / 0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php`, , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://rc728-it202-010-prod.onrender.com/project/login.php

URL
https://rc728-it202-010-prod.

URL #2
https://github.com/524RC/RC728-IT202-010/blob/prod/public_html/project/login.php

URL
https://github.com/524RC/R(

URL #3
https://github.com/524RC/RC728-IT202-010/pull/23

URL
https://github.com/524RC/R(

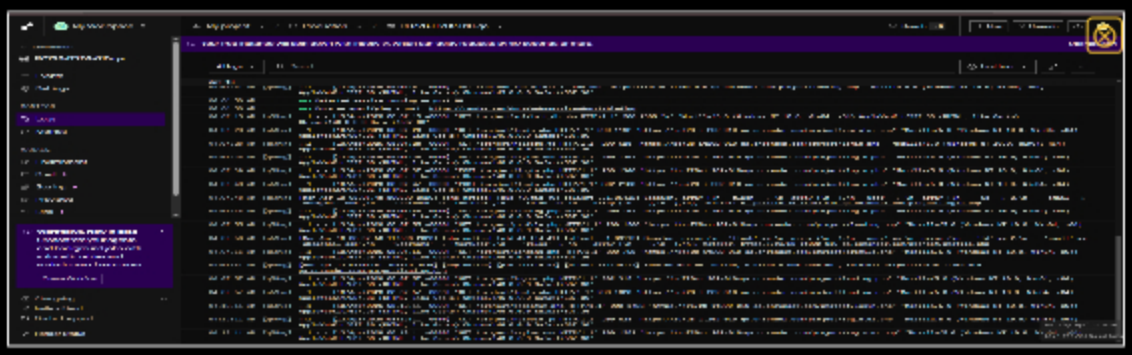
Saved: 4/13/2026 4:26:50 AM

Task #3 (0 / 0.67 pts.) - User Session Evidence

Progress: 100%

Details:

- From the render.com logs (Logs tab), capture the session error_log
- It should show the id, username, email, and roles



this is the logs for render.com

Saved: 4/13/2026 4:16:16 AM

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)
- Show the code snippet of the logout page

public_html > project > logout.php

```php
<?php
// require functions.php to pull in flash()
require(__DIR__ . "/../../lib/functions.php");
reset_session(); // clear session data and start a new session
flash("You have been logged out","success");
header("Location: " . get_url("login.php")); // redirect back to login
```

this shows the code for logout

this shows the logout message

Saved: 4/13/2026 4:29:55 AM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1
https://github.com/524RC/RC728-IT202-010/pull/23

URL
https://github.com/524RC/RC728

Saved: 4/13/2026 4:29:55 AM

Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

The logout process works by user's session data using the reset_session() function. This removed all the stored session information and starts a new sesion which basically logs the user out since their authentication data is no longer stored. After doing this, a flash message is set to appear to give a logout message to the user to inform them that they have logged out and the user is redirected back to the login page. Without the session data, the user is no longer viewed as logged in which ends their session and basically logs them out.

Saved: 4/13/2026 4:29:55 AM

Section #4: (2 pts.) Feature: Security Rules

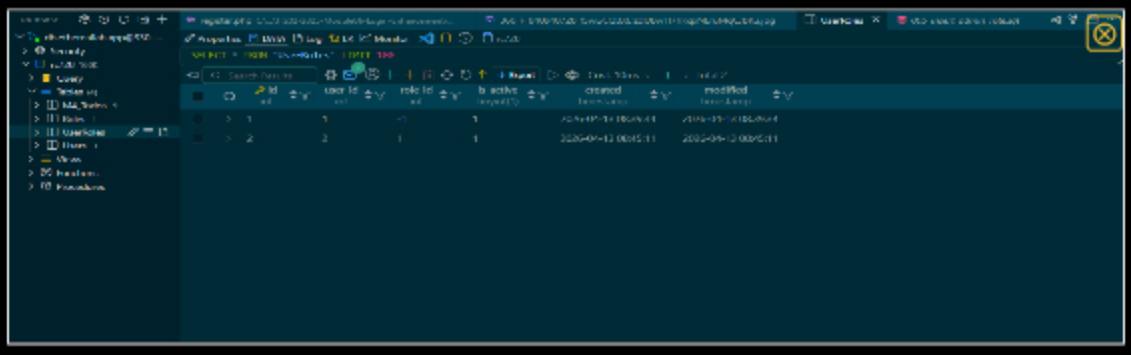
Progress: 100%

Task #1 (0 / 1 pt.) - Database Tables

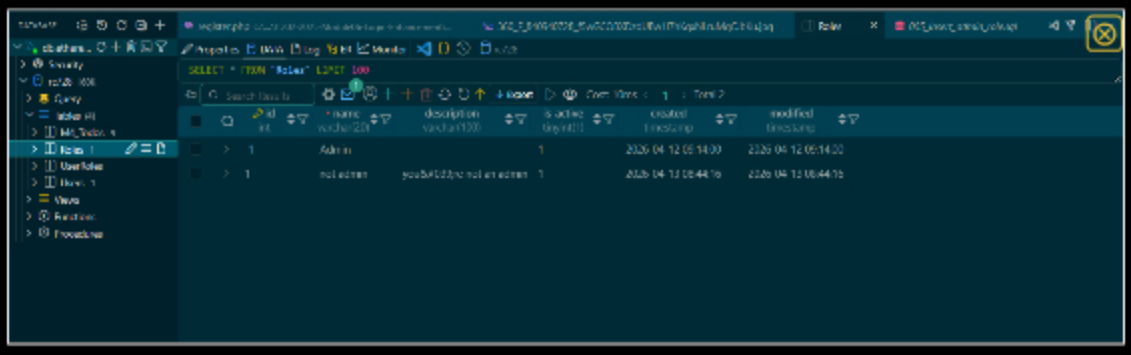
Progress: 100%

Details:

```
- From the vs code mysql tool, show both the `Roles` and `UserRoles`
```



this shows userroles



this shows roles

Saved: 4/13/2026 4:47:13 AM

Task #2 (1 pt.) - Demo Security Rules

Progress: 100%

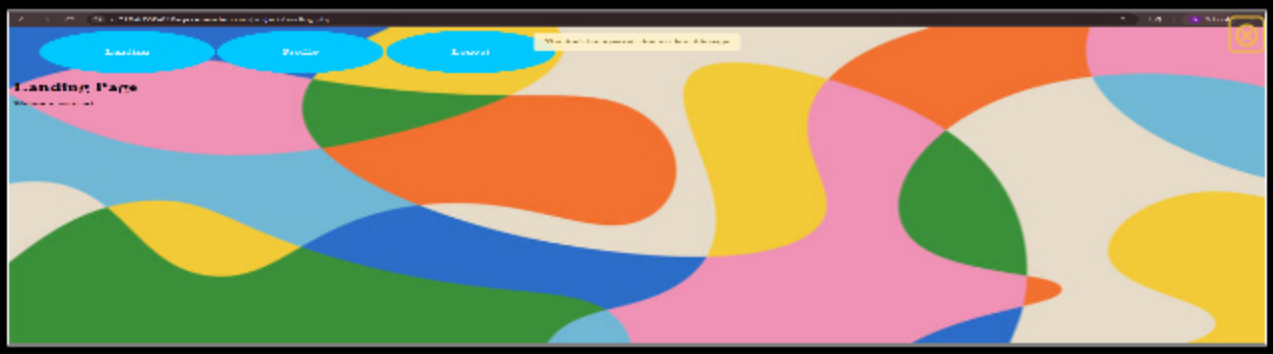
Part 1:

Progress: 100%

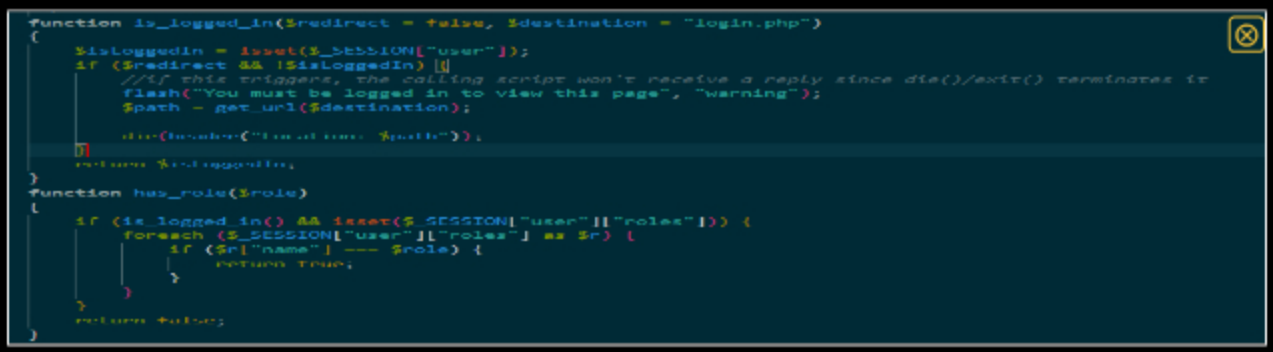
Details:

```
- Show the message related to needing to be logged in (i.e., try to manuall
- Show the message related to not having the proper permission/role (i.e.,
- Include a snippet of the login check function
- Include a snippet of the role check function
```

this shows me trying to access the landing page while logged out



this shows me trying to access a page without the permission/role



```php
function is_logged_in($redirect = false, $destination = "login.php")
{
    $isLoggedIn = isset($_SESSION["user"]);
    if ($redirect && !$isLoggedIn) {
        //if this triggers, the calling script won't receive a reply since die()/exit() terminates it
        flash("You must be logged in to view this page", "warning");
        $path = get_url($destination);

        die(header("Location: $path"));
    }
    return $isLoggedIn;
}
function has_role($role)
{
    if (is_logged_in() && isset($_SESSION["user"]["roles"])) {
        foreach ($_SESSION["user"]["roles"] as $r) {
            if ($r["name"] === $role) {
                return true;
            }
        }
    }
    return false;
}
```

this shows the code for the login check and the role check

Saved: 4/13/2026 7:34:27 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1
https://github.com/524RC/RC728-IT202-010/pull/23

URL
https://github.com/524RC/RC728

Saved: 4/13/2026 7:34:27 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

the login check works by usingthe is_logged_in() function which checks if a user session exists by looking for the $_SESSION["user"] variable. if the user is not logged in and the redirect parameter is set to true, then the function will display a flash message giving the user a warning that they aren't logged in and can't view this page. This also redirects the user to the login page. This allows for unauthenticated users to not be able to access protected pages.

The role check works by using the has_role() function which first checks if the user is logged in, then it checks what the user's role is which is stored in the user's session. It loops thorugh every role and compares them to the required role. If they match, then the function returns true allowing access to the page. If it doesn't find a match, then it returns false and prevents the user from accessing the restricted page.

Saved: 4/13/2026 7:34:27 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

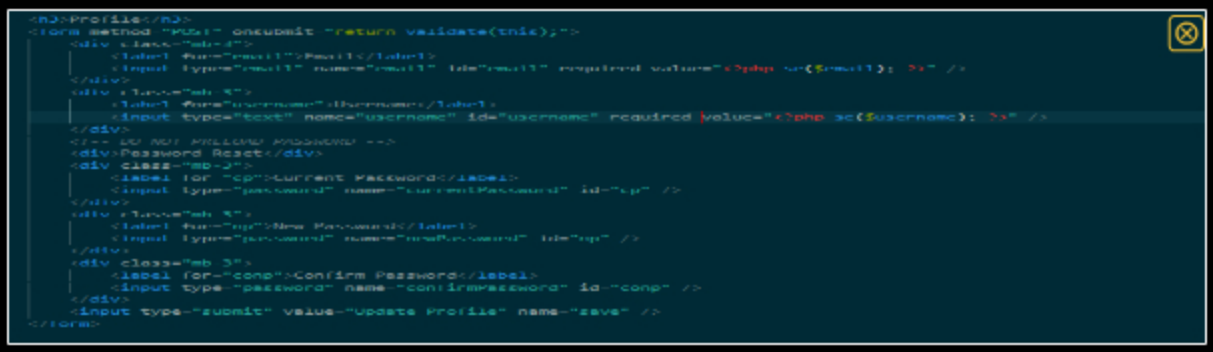
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

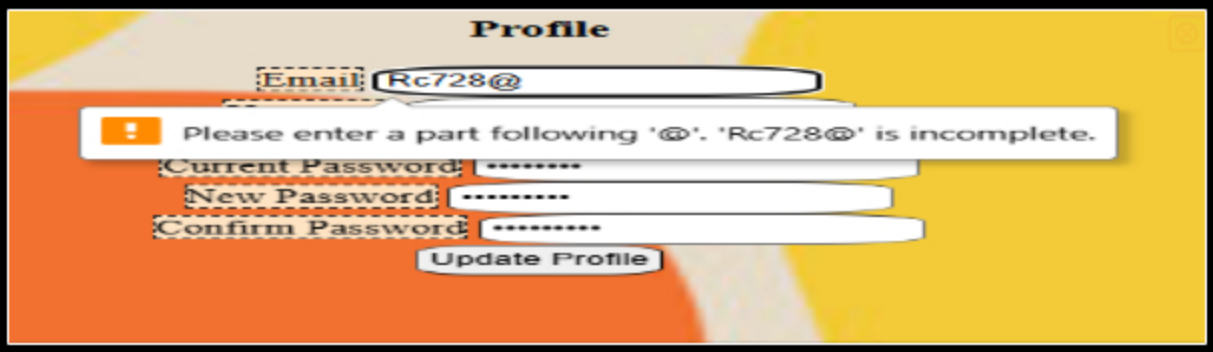
Progress: 100%

Details:

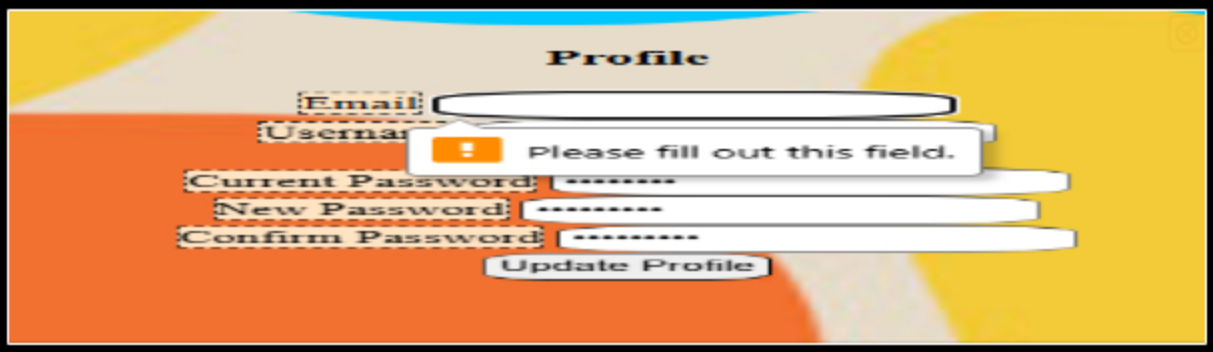
- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)



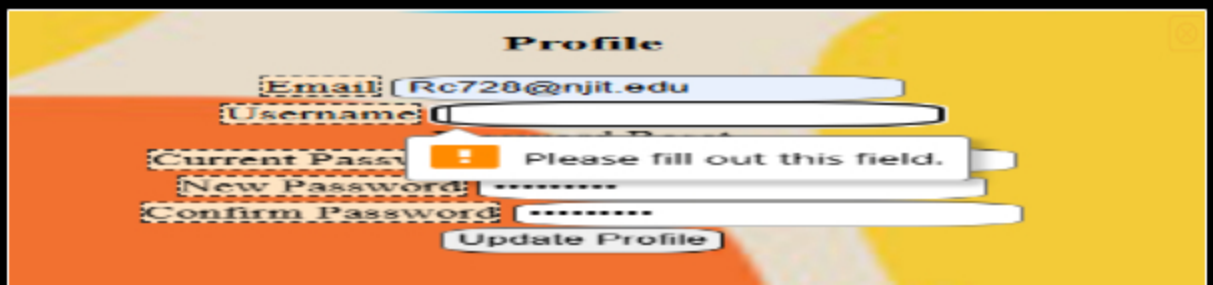
this shows the html validation code



this shows an invalid email error



this shows an empty field error for email



this shows an empty field error for username

this shows an invalid password length error

Saved: 4/13/2026 8:54:04 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The HTML validation for the profile works by using the required attribute that I added to ensure that the email and username fields aren't empty. The email field uses type="email" so the browser checks for a valid email format. The password fields use minlength="8" to make sure it has the minimum required length. These validations are handled by the browser and display these messages before submission to allow the user to fix the mistakes that they have. The form also uses sticky form logic which keep the values of the username and the email so that the user doesn't have to re-enter their information.

Saved: 4/13/2026 8:54:04 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com go after a manual

in one of few screenshots) (visit the proper file on Render.com qa after a manual deploy)

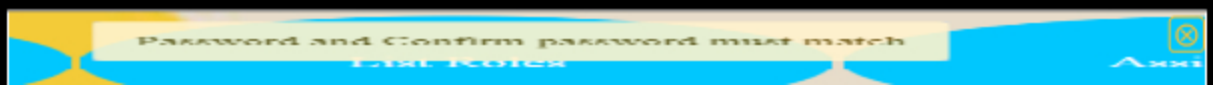
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

This shows the js code with more validation steps added

this shows invalid email format warning from js

this show invalid username error

this shows invalid password error



this shows password must match error

Saved: 4/13/2026 8:53:29 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The JS validation is handled using the validation() function which is ran when the profile form is submitted. I chose to check and validate the email and username format, password length, and if the new password matches up with the confirm password. If any of these validations end up failing, a warning message is displayed on the page and the form submission is prevented. This allows for validating user input on the client side before the form is submitted allowing the user to correct any errors made.

Saved: 4/13/2026 8:53:29 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions

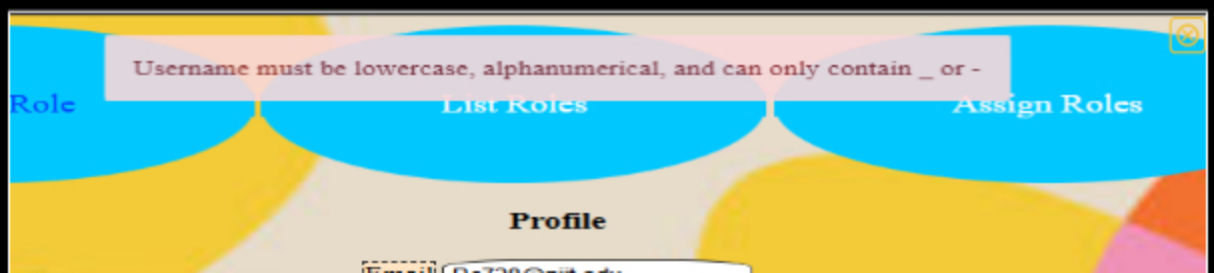
```php
if (isset($_POST["email"], $_POST["username"])) {
    $new_email = se($_POST, "email", null, false);
    $new_username = se($_POST, "username", null, false);
    $hasError = false;
    // validate format
    if (empty($new_email)) {
        //echo "Email must not be empty<br>";
        flash("Email must not be empty.", "danger");
        $hasError = true;
    }
    // sanitize and validate email
    $new_email = sanitize_email($new_email);
    if (!is_valid_email($new_email)) {
        //echo "Invalid email address<br>";
        flash("Invalid email address.", "danger");
        $hasError = true;
    }
    if (!is_valid_username($new_username)) {
        flash("Username must be lowercase, alphanumerical, and can only contain _ or -", "danger");
        $hasError = true;
    }
```

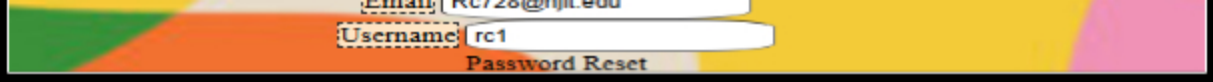
This shows the PHP validation for email and username

This shows the code for updating the user profile and handling duplicate email or username errors

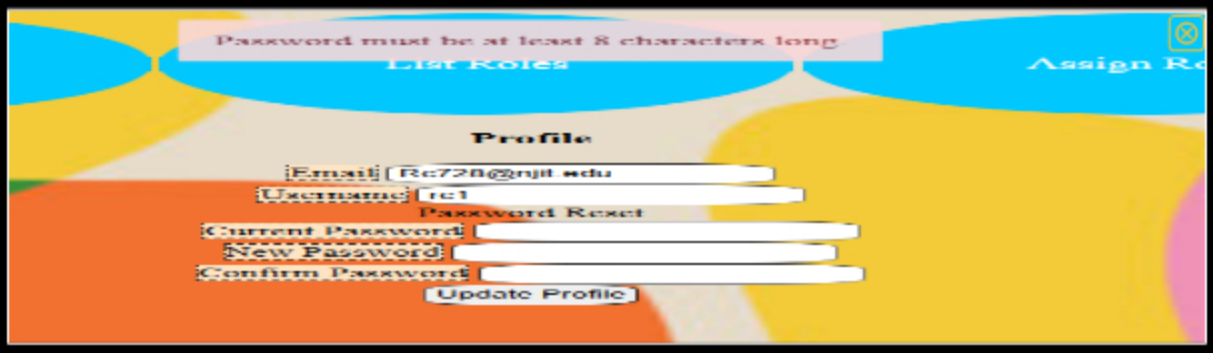
This shows the PHP validation for password updates

invalid email error





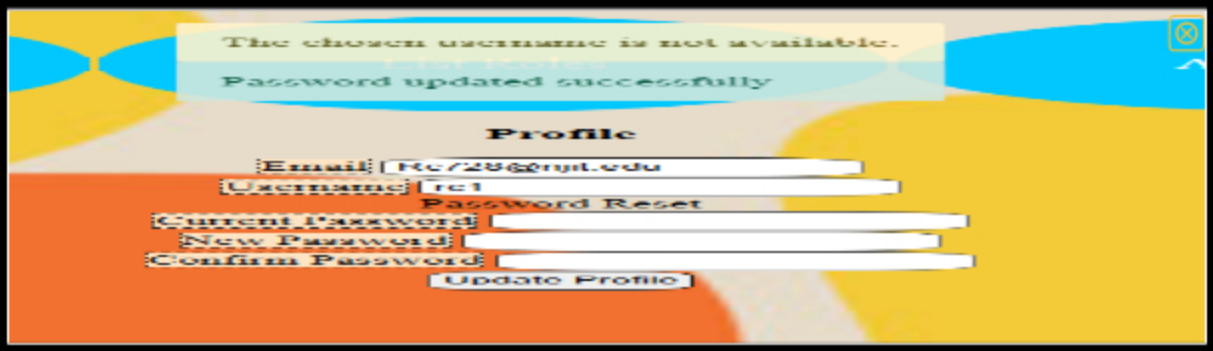
invalid username error



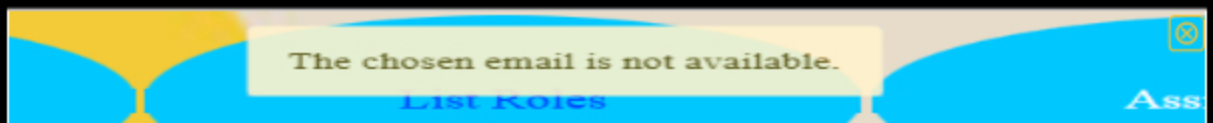
invalid password error

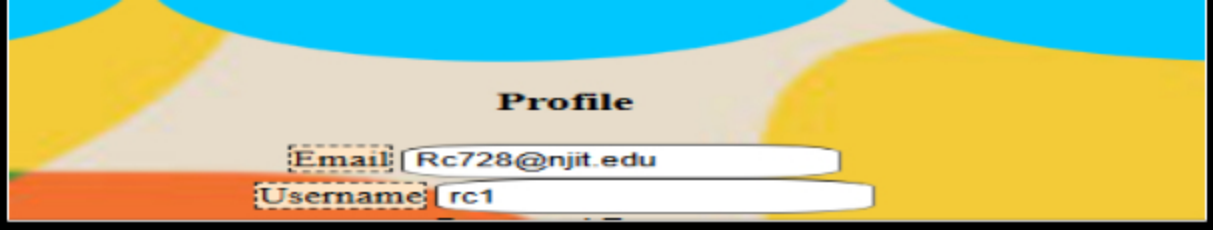
password mismatch error

invalid current password error



username taken error





email taken error

Saved: 4/13/2026 11:07:35 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The PHP validation runs after the form is submitted and before any updates that can be made to the database. I validated the email and username format, password length is a minimum of 8 characters long, new and confirm passwords match, the current password is correct, and checks if theres a duplicate username and email in the database. If any of these validations fail, a flash message will be displayed and the update is cancelled. This allows the user to correct any errors made.

Saved: 4/13/2026 11:07:35 PM

Task #2 (0 / 1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php`, , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://rc728-it202-010-prod.onrender.com/project/profile.php

URL
https://rc728-it202-010-prod.

URL #2
https://github.com/524RC/RC728-IT202-010/blob/prod/public_html/

URL
https://github.com/524RC/R(

project/profile.php

URL #3
https://github.com/524RC/RC728-IT202-010/pull/23

URL
https://github.com/524RC/R(

Saved: 4/13/2026 11:27:26 PM

Section #6: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - Github Details

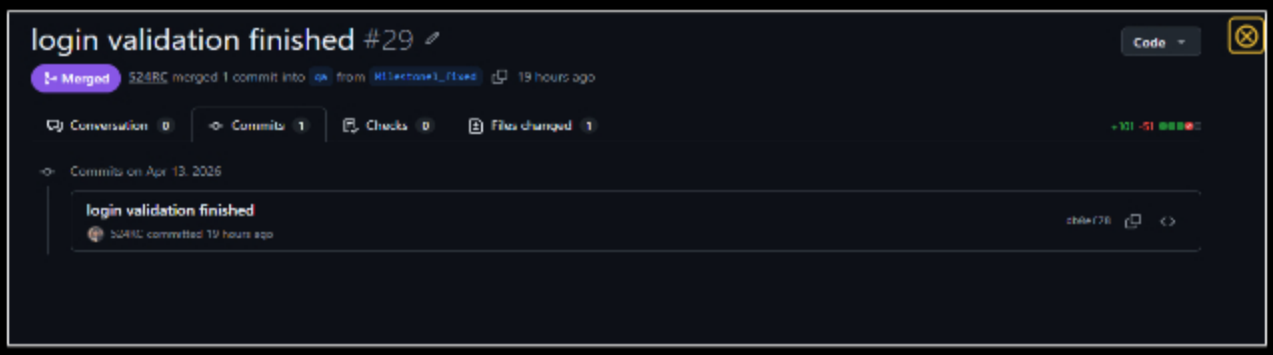
Progress: 100%

Part 1:

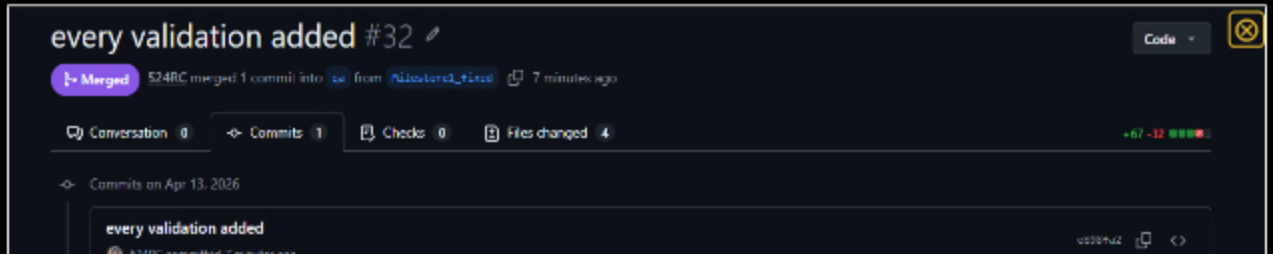
Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)

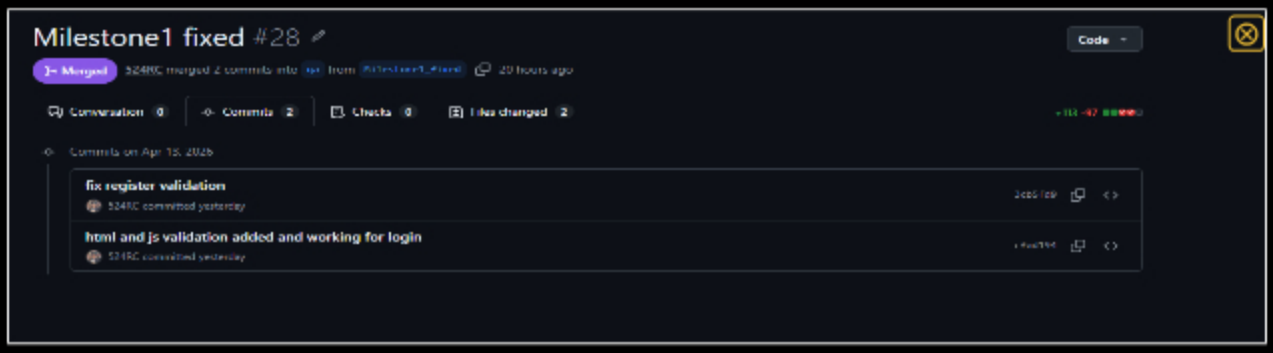
Commits tab



commits tab



commits tab



commits tab

Saved: 4/13/2026 11:31:12 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`) (Milestone1 into qa)

URL #1
https://github.com/524RC/RC728-IT202-010/pull/32/commits
URL
https://github.com/524RC/RC728

URL #2
https://github.com/524RC/RC728-IT202-010/pull/23
URL
https://github.com/524RC/RC728

URL #3
https://github.com/524RC/RC728-IT202-010/pull/29
URL
https://github.com/524RC/RC728

URL #4
https://github.com/524RC/RC728-IT202-010/pull/28
URL
https://github.com/524RC/RC728

Saved: 4/13/2026 11:31:12 PM

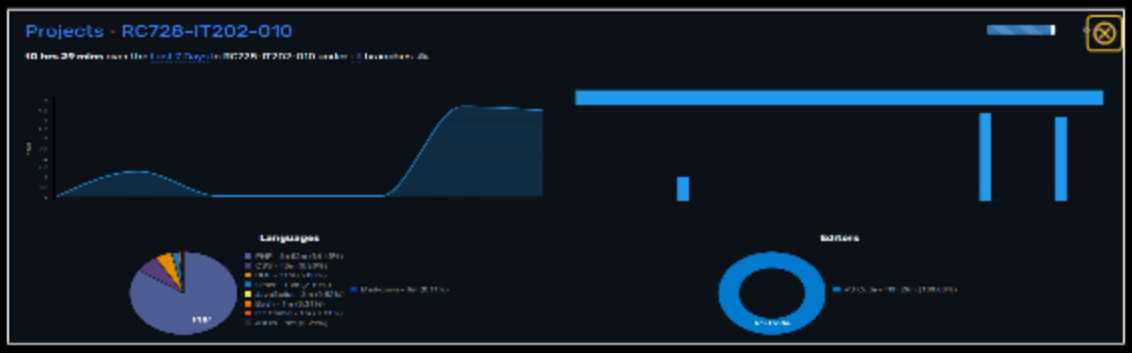
Task #2 (0 / 0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name

- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



top part of wakatime

bottom of wakatime part 1

bottom of wakatiem part 2

Saved: 4/13/2026 11:33:54 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0 / 0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned alot about validations and why each validation is needed. I also learned about security and why we have certain validations and flash messages for these validations as to not give too much information away.

Saved: 4/13/2026 11:37:17 PM

Task #2 (0 / 0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was styling the website. This part was easy and fun to do and it didn't take too much thinking to do it. A lot of the css can be found online through w3schools. Other than styling the website the other easy part to the assignment was the sql part and assigning roles through the database.

Saved: 4/13/2026 11:39:25 PM

Task #3 (0 / 0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was getting each validation to work and running into a lot of errors. doing the first validation section was hard to figure out especially the JS and making the JS. Once i did the first one it was a lot easier to do the rest since they all ran through a similar format.

Saved: 4/13/2026 11:41:10 PM